1.动态调度算法

（1）Tomasulo 算法是否能完全消除 RAW、WAR 和 WAW 相关，如果能， 请简要说明实现机制，如果不能请说明理由。（3 分）
（2）请解释什么叫精确异常？（3 分）
（3）Tomasulo 算法能否实现精确异常？为什么？

(1) 不能完全消除 RAW 相关，但可以消除 WAR 和 WAW 相关.
消除 WAR 和 WAW 相关：Tomasulo 算法通过寄存器重命名来消除 WAR 和 WAW 相关。在指令发射阶段，会对指令进行寄存器重命名，将指令中的逻辑寄存器映射到物理寄存器或保留站，通过保留站和重排序缓冲器来实现，从而避免了 WAR 和 WAW 冲突。

(2) 精确异常是指当指令出现异常时，如除 0、页面错误等，前面的指令已经完成，后面的指令不能对寄存器、内存等进行修改，即跟顺序执行的效果一样。也就是说，在精确异常的情况下，处理器的状态就如同所有指令都是按程序顺序执行到异常指令处时的状态。

(3) 基础的 Tomasulo 算法不能实现精确异常，带有 Reorder Buffer 的 Tomasulo 算法可以实现精确异常。基础的 Tomasulo 算法由于指令乱序执行，无法保证在异常发生时，前面的指令都已执行完成，后面的指令未对系统状态进行修改。

2. 性能分析

假设有一个长流水线，仅仅对条件转移指令使用分支目标缓冲（BTB）。此外，假设分支预测错误的开销为 4 个时钟周期，缓存缺失的开销为 3 个时钟周期，缓 存命中率为 90%，分支预测精度为 90%，分支指令频率为 15%，没有分支的基本 CPI 为 1。根据这些条件，计算：
（1）求程序执行的 CPI。（3 分）
（2）相对于采用固定的 2 个时钟周期延迟的分支处理，哪种方法程序执行速度 更快？

(1) 基本 CPI:1
分支预测错误的额外 CPI:0.15*0.1*4= 0.06
缓存缺失的额外 CPI：0.1*3= 0.3
综上，总 CPI = 基本 CPI + 分支额外 CPI + 缓存额外 CPI=1+0.06+0.3= 1.36
(2) 固定 2 周期延迟的分支额外 CPI：无论分支是否成功，所有分支指令都固定增加 2 个时钟周期延迟，计算方式为 “分支指令频率 × 固定延迟”。
分支额外 CPI=0.15×2= 0.3
总 CPI = 基本 CPI + 固定分支额外 CPI + 缓存额外 CPI=1+0.3+0.3= 1.6

由于 1.36＜1.6，采用 BTB 的分支处理方法程序执行速度更快。

3. 相关与冒险

请阐述 MIPS 的 5 段流水线如何检测数据相关所造成的冒险？（4 分）为什么名相关对于顺序执行的 5 段 MIPS 流水线来说不会造成冒险？（2 分）

(1) 数据前推：优先处理可加速的情况,直接将未写回寄存器的数据“提前送达”执行阶段，避免插入气泡。
插入气泡：处理无法前推的特殊情况,在当前指令的 IF 和 ID 阶段之间插入 1 个气泡（即当前指令的 IF 阶段延迟 1 周期），同时暂停前指令之后的流水线，确保数据写回后再读取。
(2) 名相关分为 WAR 和 WAW，核心是“寄存器名重复但无实际数据依赖”。在顺序执行的 5 段流水线中，寄存器的读写顺序严格遵循指令顺序, 导致名相关无法形成冒险

4. 缓存一致性协议

请对比分析缓存一致性协议中写更新和写无效两种方案的优缺点。（4 分）
请分析目录一致性协议的优点，和存在的问题。（4 分）

(1) 写更新和写无效方案的优缺点对比
写更新方案：优点是一旦某缓存缓存了某一个共享变量，它就会一直拥有此变量的最新备份，无需像写无效方案那样因数据无效而重新获取，减少了缓存缺失的次数。缺点是当某一共享变量被更新时，所有拥有该变量的处理器核的备份都会被更新，哪怕这些处理器已经不需要这个变量了，这会带来不必要的带宽消耗和处理开销。

写无效方案：优点是当一个处理器核使某一变量在所有拥有该备份的缓存中无效后，它就取得了对此变量的独占权，随后对此变量的更新不必再通知其它处理器核，直到其它处理器核请求访问此变量而导致独占权被取消，减少了后续更新的通信开销。缺点是当某变量在一个处理器核中的备份无效后，此处理器核再访问该变量会引起缓存缺失，当一个共享块被多个处理器核频繁访问时会导致 “乒乓” 效应，即数据在不同处理器核的缓存之间频繁无效和重新获取，导致性能严重下降。

(2)目录一致性协议的优点和问题

优点：一是可扩展性好，适合大规模多处理器系统，通过目录可以有针对性地通知相关缓存进行状态更新，而不是像监听协议那样广播给所有处理器，有效减少了总线流量。二是一致性维护效率高，能够更及时地更新其他缓存中的数据，减少一致性维护延迟，因为目录可以精确控制数据块状态的更新，避免了不必要的广播。

问题：目录一致性协议的实现相对复杂，需要额外的存储空间和处理器资源来维护目录结构。目录格式设计需要考虑存储代价和信息溢出等问题，如全位向量目录格式存储代价大，受限指针目录格式可能会出现指针上限不足的情况。此外，目录的位置设计也较为复杂，集中式目录可能会成为系统瓶颈，分布式目录的设计则面临诸多挑战。